

AGENTS AND PARALLEL WORK

Create custom subagents

 Copy page

Create and use specialized AI subagents in Claude Code for task-specific workflows and improved context management.

Subagents are specialized AI assistants that handle specific types of tasks. Use one when a side task would flood your main conversation with search results, logs, or file contents you won't reference again: the subagent does that work in its own context and returns only the summary. Define a custom subagent when you keep spawning the same kind of worker with the same instructions.

Each subagent runs in its own context window with a custom system prompt, specific tool access, and independent permissions. When Claude encounters a task that matches a subagent's description, it delegates to that subagent, which works independently and returns results. To see the context savings in practice, the [context window visualization](#) walks through a session where a subagent handles research in its own separate window.

❗ Subagents work within a single session. To run many independent sessions in parallel and monitor them from one place, see [background agents](#). For separate sessions that pass messages to each other, see [cross-session messaging](#). For a coordinated team of sessions Claude spawns and supervises, see [agent teams](#).

Subagents help you:

- **Preserve context** by keeping exploration and implementation out of your main conversation
- **Enforce constraints** by limiting which tools a subagent can use
- **Reuse configurations** across projects with user-level subagents
- **Specialize behavior** with focused system prompts for specific domains

Those descriptions take up context, so keep them short. When the combined descriptions of your subagents, except the built-in ones, exceed 15,000 tokens, Claude Code shows a **warning at startup with the total token count**. Trim the `description` fields of your subagents, and move detail into each subagent's system prompt, which only loads when that subagent runs.

Built-in subagents

Claude Code includes built-in subagents that Claude automatically uses when appropriate. Each inherits the parent conversation's permissions; most run with a restricted tool set.

Explore and Plan skip your CLAUDE.md files and the parent session's git status to keep research fast and inexpensive. Every other built-in and **custom subagent** loads both. For the full breakdown of what reaches a subagent, see **what loads at startup**.

Explore **Plan** **General-purpose** **Other**

A fast, read-only agent optimized for searching and analyzing codebases.

- **Model:** inherits from the main conversation, capped at Opus on the Claude API, so Explore never runs on a more expensive model than the one you already chose for the session
- **Tools:** read-only tools; Write and Edit are denied
- **Purpose:** file discovery, code search, codebase exploration

As of v2.1.198, Explore inherits the main conversation's model instead of always running on Haiku. On the Claude API, the inherited model is capped at Opus: a main conversation on a higher tier runs Explore on Opus, and a main conversation on Sonnet or Haiku runs Explore on that same model. On any other provider, such as **Amazon Bedrock, Google**

`model` field, so define one with `model: llama` to keep exploration on a lower cost model.

Claude delegates to Explore when it needs to search or understand a codebase without making changes. This keeps exploration results out of your main conversation context.

When invoking Explore, Claude specifies a thoroughness level: **quick** for targeted lookups, **medium** for balanced exploration, or **very thorough** for comprehensive analysis.

Built-in subagents are registered by default in interactive sessions. To restrict them:

- To block a specific built-in type, add it to `permissions.deny` as shown in [Disable specific subagents](#).
- To prevent Claude from delegating to any subagent, deny the `Agent` tool itself with `permissions.deny`.
- To remove only the built-in `Explore` and `Plan` subagents, set `CLAUDE_CODE_DISABLE_EXPLORE_PLAN_AGENTS=1`. Claude reads and explores files directly instead of delegating to them. Requires Claude Code v2.1.198 or later.
- In [non-interactive mode](#) and the [Agent SDK](#), set `CLAUDE_AGENT_SDK_DISABLE_BUILTIN_AGENTS=1` to remove all built-in types and supply only your own.

An Agent tool call that omits `subagent_type` fails with `subagent_type is required` when the session has no `general-purpose` subagent to fall back on.

Beyond these built-in subagents, you can create your own with custom prompts, tool restrictions, permission modes, hooks, and skills. The following sections show how to get started and customize subagents.

As of v2.1.198, the `/agents` command no longer opens the interactive creation wizard; running it prints a reminder to ask Claude or edit `.claude/agents/` directly. Subagent files, frontmatter fields, and the `.claude/agents/` and `~/.claude/agents/` locations are unchanged; only the terminal wizard is removed.

This walkthrough creates a user-level subagent that reviews code and suggests improvements.

1 Ask Claude to create the subagent

In Claude Code, describe the subagent you want and where to save it:

```
Create a personal code-improver subagent in ~/.claude/agents/ that
scans
files and suggests improvements for readability, performance, and
best
practices. It should explain each issue, show the current code, and
provide an improved version. Make it read-only and have it use
Sonnet.
```

Claude writes the file with a `name`, a `description`, a `tools` list, a `model`, and a system prompt.

2 Review the file

Open `~/.claude/agents/code-improver.md` and confirm the frontmatter matches what you asked for. The result looks like this:

```
model: sonnet
```

```
---
```

You are a code improvement specialist. For each issue you find, explain the problem, show the current code, and provide an improved version.

Because the file lives in `~/.claude/agents/`, the subagent is available in every project on your machine. To scope it to one project instead, move it to that project's `.claude/agents/` directory. **Choose the subagent scope** compares the two.

3 Try it out

Ask Claude to delegate to the new subagent:

```
Use the code-improver agent to suggest improvements in this project
```

Claude delegates to your new subagent, which scans the codebase and returns improvement suggestions. In the transcript, the delegation appears as a tool call row showing the subagent's name followed by a short task description, such as `code-improver(Suggest code improvements)`.

If Claude can't find the new subagent, restart Claude Code and try again. This happens only when `~/.claude/agents/` didn't exist before the session started, because a running session doesn't detect a newly created `agents` directory.

You now have a subagent you can use in any project on your machine to analyze codebases and suggest improvements.

You can also write subagent files by hand, define them via CLI flags, or distribute them through plugins. The following sections cover all configuration options.

Configure subagents

A subagent's file location determines who it's available to, and its frontmatter determines what it can do. This section covers where subagent files live and every field they support.

Choose the subagent scope

Store subagent files in different locations depending on scope. When multiple subagents share the same name, Claude Code uses the one from the higher-priority location.

Location	Scope	Priority	How to create
Managed settings	Organization-wide	1 (highest)	Deployed via <u>managed settings</u>
<code>--agents</code> CLI flag	Current session	2	Pass JSON when launching Claude Code
<code>.claude/agents/</code>	Current project	3	Ask Claude, or create the file manually
<code>~/.claude/agents/</code>	All your projects	4	Ask Claude, or create the file manually
Plugin's <code>agents/</code> directory	Where plugin is enabled	5 (lowest)	Installed with <u>plugins</u>

Project subagents (`.claude/agents/`) are ideal for subagents specific to a codebase. Check them into version control so your team can use and improve them collaboratively.

Project subagents are discovered by walking up from the current working directory, so every `.claude/agents/` between there and the repository root is scanned. As of v2.1.178, when more than one of these nested directories defines the same `name` , Claude Code uses the definition closest to the working directory.

User subagents (`~/.claude/agents/`) are personal subagents available in all your projects.

Claude Code scans `.claude/agents/` and `~/.claude/agents/` recursively, so you can organize definitions into subfolders such as `agents/review/` or `agents/research/` . The subdirectory path doesn't affect how a subagent is identified or invoked, because identity comes only from the `name` frontmatter field.

Keep `name` values unique across the whole tree: if two files under the same `.claude/agents/` directory, including its subfolders, declare the same name, Claude Code loads only one of them, chosen by filesystem read order rather than a documented precedence. Across nested project directories, the definition closest to the working directory wins, as described above. The `/doctor` setup checkup reports files in the same directory that share a name and proposes renaming or removing all but one. Before v2.1.205, `/doctor` opened a diagnostics screen that listed duplicates and showed which definition was active.

Plugin `agents/` directories are also scanned recursively. Unlike project and user scopes, a subfolder inside a plugin's `agents/` directory becomes part of the **scoped identifier**: a file at `agents/review/security.md` in plugin `my-plugin` registers as `my-plugin:review:security` .

CLI-defined subagents are passed as JSON when launching Claude Code. They exist only for that session and aren't saved to disk, making them useful for quick testing or automation scripts. You can define multiple subagents in a single `--agents` call:

macOS, Linux, WSL Windows PowerShell

```
"tools": ["Read", "Grep", "Glob", "Bash"],
"model": "sonnet"
},
"debugger": {
  "description": "Debugging specialist for errors and test failures.",
  "prompt": "You are an expert debugger. Analyze errors, identify root causes, and provide solutions."
}
}'
```

The `--agents` flag accepts JSON with a `prompt` field plus these **frontmatter** fields: `description`, `tools`, `disallowedTools`, `model`, `permissionMode`, `mcpServers`, `hooks`, `maxTurns`, `skills`, `initialPrompt`, `memory`, `effort`, `background`, and `isolation`. Use `prompt` for the system prompt, equivalent to the markdown body in file-based subagents.

Managed subagents are deployed by organization administrators. Place markdown files in `.claude/agents/` inside the **managed settings directory**, using the same frontmatter format as project and user subagents. Managed definitions take precedence over project and user subagents with the same name.

Plugin subagents come from **plugins** you've installed. They load automatically alongside your custom subagents and appear in the @-mention typeahead under their scoped name. See the **plugin components reference** for details on creating plugin subagents.

❗ For security reasons, plugin subagents don't support the `hooks`, `mcpServers`, or `permissionMode` frontmatter fields. These fields are ignored when loading agents from a plugin. If you need them, copy the agent file into `.claude/agents/` or `~/.claude/agents/`. You can also add rules to `permissions.allow` in `settings.json` or `settings.local.json`, but these rules apply to the entire session, not only the plugin subagent.

Subagent definitions from any of these scopes are also available to **agent teams**: when spawning a teammate, you can reference a subagent type, and Claude Code applies

Subagent files use YAML frontmatter for configuration, followed by the system prompt in Markdown:

- ❗ Claude Code watches `~/.claude/agents/` and `.claude/agents/`. When you add or edit a subagent file on disk, or ask Claude to write one for you, Claude Code detects the change within a few seconds and the next delegation uses the updated definition, with no restart needed.

Three cases still need a restart:

- The watcher covers only directories that existed when the session started, so after creating a scope's first agent file in a new `agents` directory, restart to load it.
- Claude Code doesn't watch `.claude/agents/` inside directories added with `--add-dir` or `/add-dir`, so after adding or editing a subagent there, restart to load the change.
- Sessions started with `--disable-slash-commands` don't watch these directories at all.

`.claude/agents/code-reviewer.md`

```
---
name: code-reviewer
description: Reviews code for quality and best practices
tools: Read, Glob, Grep
model: sonnet
---
```

You are a code reviewer. When invoked, analyze the code and provide specific, actionable feedback on quality, security, and best practices.

The frontmatter defines the subagent's metadata and configuration. The body becomes the system prompt that guides the subagent's behavior. Subagents receive only this system prompt plus basic environment details like the working directory, not the full Claude Code system prompt.

A subagent starts in the main conversation's current working directory. Within a subagent, `cd` commands don't persist between Bash or PowerShell tool calls and don't affect the main conversation's working directory. To give the subagent an isolated copy of the repository instead, set `isolation: worktree`.

A subagent with `isolation: worktree` runs its Bash and PowerShell commands inside its worktree. A command whose working directory resolves to your main checkout instead, for example because the worktree directory was removed while the subagent was running, fails with an error. Before v2.1.203, such a command could run in the main checkout.

This working-directory check covers the whole repository containing the directory you launched Claude Code from. When your session runs in a linked worktree of its own, the check also covers the main checkout that worktree is linked from. Before v2.1.210, the check covered only the launch directory itself. A command whose working directory resolved elsewhere in the same repository, such as the repository root when you launched Claude Code from a monorepo subdirectory, ran there instead of failing.

For Bash commands, Claude Code also checks the command itself in two ways:

- It blocks a command that redirects git into the main checkout.
- It refuses a command whose shape it can't verify stays inside the worktree. This refusal applies even to a command that runs no git.

The redirect vectors and the shape rules are listed under **How Claude Code enforces isolation**. PowerShell commands get only the working-directory check.

Monitor commands go through the same working-directory and command-content checks as Bash commands.

When the main conversation itself runs isolated in a worktree, Claude Code applies the same checks to the session and to every subagent it spawns, including subagents

`description` are required.

Field	Required	Description
<code>name</code>	Yes	Unique identifier using lowercase letters and hyphens. Hooks receive this value as <code>agent_type</code> . The filename doesn't have to match. Names can't contain <code>:</code> , which is reserved for plugin-scoped identifiers such as <code>my-plugin:reviewer</code> . Claude Code doesn't load a file whose name contains one and logs an error to the debug log. Before v2.1.218, such names were accepted
<code>description</code>	Yes	When Claude should delegate to this subagent
<code>tools</code>	No	Tools the subagent can use. Inherits every tool available to subagents if omitted. If no entry in the list resolves to a tool, the subagent usually fails to launch with an error naming the entries. To preload Skills into context, use the <code>skills</code> field rather than listing <code>Skill</code> here
<code>disallowedTools</code>	No	Tools to deny, removed from inherited or specified list
<code>model</code>	No	Model to use: <code>sonnet</code> , <code>opus</code> , <code>haiku</code> , <code>fable</code> , a full model ID (for example, <code>claude-opus-5</code>), or <code>inherit</code> . Defaults to <code>inherit</code>
<code>permissionMode</code>	No	Permission mode: <code>default</code> , <code>acceptEdits</code> , <code>auto</code> , <code>dontAsk</code> , <code>bypassPermissions</code> , <code>plan</code> , or <code>manual</code> as an alias for <code>default</code> . The <code>manual</code> alias requires Claude Code v2.1.200 or later. Ignored for plugin subagents
<code>maxTurns</code>	No	Maximum number of agentic turns before the subagent stops. When the subagent reaches the limit, Claude Code returns its output marked as partial, and Claude can resume it to continue. The partial marking requires Claude Code v2.1.246 or later

mcpServers	No	MCP servers available to this subagent. Each entry is either a server name referencing an already-configured server (e.g., "slack") or an inline definition with the server name as key and a full MCP server config as value. Ignored for plugin subagents
hooks	No	Lifecycle hooks scoped to this subagent. Ignored for plugin subagents
memory	No	Persistent memory scope: user , project , or local . Enables cross-session learning
background	No	Set to true to keep this subagent in the background even when Claude asks to run it in the foreground. Where fork mode is on, Claude Code already runs the subagents Claude spawns in the background
effort	No	Effort level when this subagent is active. Overrides the session effort level. Default: inherits from session. Options: low , medium , high , xhigh , max ; available levels depend on the model
isolation	No	Set to worktree to run the subagent in a temporary git worktree , giving it an isolated copy of the repository branched by default from your default branch rather than the parent session's HEAD . The worktree is automatically cleaned up if the subagent makes no changes
color	No	Display color for the subagent in the task list and transcript. Accepts red , blue , green , yellow , purple , orange , pink , or cyan
initialPrompt	No	Auto-submitted as the first user turn when this agent runs as the main session agent (via --agent or the agent setting). Commands and skills are processed. Prepend to any user-provided prompt

Subagent files Claude Code skips

Claude Code skips a file in a project, user, or managed agents directory, or in one under a directory you add with --add-dir , without reporting it in the session, when the

an error to the debug log. See the `name` row in the table above.

- **A `name` but no `description`** : Claude Code skips the file and writes the reason to the debug log.
- **YAML that doesn't parse**: Claude Code reads no fields from the file, skips it, and writes the parse error to the debug log.

To see the debug log, run Claude Code with `--debug` .

A **plugin subagent** whose frontmatter has no `name` or doesn't parse still loads, under its filename.

Check an `agents` directory before a session

To find files in an `agents` directory whose frontmatter doesn't parse, run `claude plugin validate` against the directory, for example `.claude/agents` or `~/.claude/agents` . Claude Code checks only **the directory you name**, and doesn't flag a file whose frontmatter parses but has no `name` . Requires Claude Code v2.1.233 or later.

Choose a model

The `model` field controls which **AI model** the subagent uses:

- **Model alias**: use one of the available aliases: `sonnet` , `opus` , `haiku` , or `fable`
- **Full model ID**: use a full model ID such as `claude-opus-5` or `claude-sonnet-5` . Accepts the same values as the `--model` flag
- **inherit**: use the same model as the main conversation
- **Omitted**: defaults to `inherit` and uses the same model as the main conversation

When Claude invokes a subagent, it can also pass a `model` parameter for that specific invocation. Claude Code resolves the subagent's model in this order:

4. The main conversation's model

As of v2.1.196, setting `CLAUDE_CODE_SUBAGENT_MODEL` to `inherit` is the same as leaving it unset: resolution continues with the per-invocation `model` parameter, then the frontmatter. In earlier versions, `inherit` forced subagents onto the main conversation's model and ignored both of those sources.

Claude Code checks the environment variable, per-invocation parameter, and frontmatter values against your organization's `availableModels` allowlist. For a blocked value, it substitutes another model:

- When the blocked value is a family alias such as `opus`, Claude Code runs the subagent on the newest version of that family the allowlist permits, following the same **substitution rules and provider scope** as `/model`. Before v2.1.222, Claude Code ran the subagent on the inherited model for a blocked family alias as well.
- For any other blocked value, on providers where that substitution doesn't operate, or when the allowlist permits no version of the family, Claude Code runs the subagent on the inherited model instead.

In interactive sessions, Claude Code shows a warning naming the requested model and the model the subagent runs on, for either substitution.

A per-invocation `model` parameter also applies when the subagent is **resumed or sent a follow-up message**, so the subagent stays on that model. Before v2.1.211, resuming dropped the per-invocation value and the subagent reverted to its definition's `model` field or, without one, the main conversation's model.

As of v2.1.198, subagents also inherit the main conversation's **extended thinking** configuration: if thinking is on in your session, it's on for the subagent, and if it's off, it stays off. There is no per-subagent thinking setting. Before v2.1.198, subagents ran with extended thinking disabled regardless of the main conversation's setting.

Available tools

Subagents inherit the **built-in tools** and MCP tools available in the main conversation, narrowed by two filters: the first removes a short list of tools from every subagent, and the second reduces the built-in tool set for subagents that run in the **background**, which is the default. **Forks** skip both filters and receive the main conversation's exact tool pool. The first filter removes these tools, even when listed in the `tools` field:

- `Agent` , when the subagent is at the **depth limit**; in a **fork** the tool stays listed but returns an error instead of spawning
- `AskUserQuestion`
- `EndConversation` , which can end only the main conversation; see **EndConversation tool behavior**
- `EnterPlanMode`
- `ExitPlanMode` , unless the subagent's **permissionMode** is `plan`
- `ScheduleWakeup`
- `TaskOutput`
- `WaitForMcpServers`
- `Workflow`

The second filter applies to subagents running in the background. Apart from `Agent` and `ExitPlanMode` , which follow the first filter's conditions wherever the subagent runs, a background subagent keeps every MCP tool but only these built-in tools: `Read` , `Grep` , `Glob` , `Bash` , `PowerShell` , `Edit` , `Write` , `NotebookEdit` , `WebFetch` , `WebSearch` , `TodoWrite` , `Skill` , `ToolSearch` , `EnterWorktree` , `ExitWorktree` , `Monitor` , `TaskStop` , `SendMessage` , and `Artifact` . Claude Code removes every other built-in tool from a background subagent, whether inherited or listed in the `tools` field, so the same definition can resolve to different tools in the foreground and the background. The removal reports no error unless it leaves the `tools` list **resolving to nothing**. `ListAgents` follows these filters like any built-in tool: a foreground subagent

`taskset`, `tasklist`, `taskupdate`, `clonecreate`, `clonedelete`, and `clonelist`.

In a **session without the Task tools**, Claude Code doesn't provide the task tools to subagents either, even when the subagent runs a different model. An in-process teammate follows your session the same way, while a teammate in its own **split pane** runs as a separate Claude Code process, so its own model decides.

To restrict tools, use the `tools` field as an allowlist or the `disallowedTools` field as a denylist. This example uses `tools` to allow only Read, Grep, Glob, and Bash. The subagent can't edit files, write files, or use any MCP tools:

```
---
name: safe-researcher
description: Research agent with restricted capabilities
tools: Read, Grep, Glob, Bash
---
```

This example uses `disallowedTools` to inherit the subagent's tool pool except Write and Edit. The subagent keeps Bash, MCP tools, and the rest of its pool:

```
---
name: no-writes
description: Inherits the available tools except file writes
disallowedTools: Write, Edit
---
```

If both are set, `disallowedTools` is applied first, then `tools` is resolved against the remaining pool. A tool listed in both is removed.

When nothing in the `tools` list resolves to a tool, for example because every entry is misspelled or names a tool that isn't available to subagents, Claude Code usually refuses to launch the subagent and the Agent tool returns an error naming the unresolved

Both fields accept MCP server-level patterns in addition to exact tool names.

`mcp__<server>` or `mcp__<server>__*` grants or removes every tool from the named server. In `disallowedTools`, `mcp__*` also removes every MCP tool from any server. This example removes every tool from the `github` MCP server while keeping tools from other servers and the built-in tools in its pool:

```
---
name: local-only
description: Inherits every tool except those from the github MCP server
disallowedTools: mcp__github
---
```

Restrict which subagents can be spawned

When an agent runs as the main thread with `claude --agent`, it can spawn subagents using the Agent tool. To restrict which subagent types it can spawn, use

`Agent(agent_type)` syntax in the `tools` field.

❗ In version 2.1.63, the Task tool was renamed to Agent. Existing `Task(...)` references in settings and agent definitions still work as aliases.

```
---
name: coordinator
description: Coordinates work across specialized agents
tools: Agent(worker, researcher), Read, Bash
---
```

This is an allowlist: only the `worker` and `researcher` subagents can be spawned. If the agent tries to spawn any other type, the request fails and the agent sees only the allowed types in its prompt. To block specific agents while allowing all others, use `permissions.deny` instead.

If you omit `Agent` from the `tools` list entirely, the agent can't spawn any subagents with the Agent tool.

The `Agent(agent_type)` allowlist syntax applies only to an agent running as the main thread with `claude --agent`. In a subagent definition, listing `Agent` in `tools` lets that subagent spawn subagents of its own while the depth limit allows it, but any type list inside the parentheses is ignored.

Scope MCP servers to a subagent

Use the `mcpServers` field to give a subagent access to **MCP** servers that aren't available in the main conversation. Inline servers defined here are connected when the subagent starts, subject to the trust rule for the agent file's folder, and disconnected when it finishes. String references share the parent session's connection.

❗ The `mcpServers` field applies in both contexts where an agent file can run:

- As a subagent, spawned through the Agent tool or an @-mention
- As the main session, launched with `--agent` or the `agent` setting

When the agent is the main session, inline server definitions connect at startup alongside servers from `.mcp.json` and settings files, under the same trust rule for the agent file's folder. In `/mcp`, a remote (HTTP or SSE) server you've used before can show the cached status instead; Claude Code connects it when Claude first calls one of its tools.

Each entry in the list is either an inline server definition or a string referencing an MCP server already configured in your session:

```
# inline definition: scoped to this subagent only
- playwright:
  type: stdio
  command: npx
  args: ["-y", "@playwright/mcp@latest"]
# Reference by name: reuses an already-configured server
- github
---
```

Use the Playwright tools to navigate, screenshot, and interact with pages.

Inline definitions use the same schema as `.mcp.json` server entries, keyed by the server name, and support the `stdio`, `http`, `sse`, and `ws` types.

To keep an MCP server out of the main conversation entirely and avoid its tool descriptions consuming context there, define it inline here rather than in `.mcp.json`. The subagent gets the tools; the parent conversation doesn't.

Claude Code loads an inline server from an agent file in your project's `.claude/agents/` directory, or in an `--add-dir` directory's `.claude/agents/`, only after you **trust the folder the agent file came from**. Before v2.1.238, Claude Code loaded these servers without checking trust.

- **Trust that doesn't count:** a parent folder's trust, and the automatic trust a `-p` or SDK session gets for **hooks in settings files**
- **Until then:** Claude Code skips every inline server in that agent file and writes the exact `projects["<path>"].hasTrustDialogAccepted` key for `~/.claude.json` to the debug log
- `--add-dir` **directories:** a directory outside your trusted workspace's repository needs its own trust entry, since its `.claude/agents/` files don't inherit your workspace's trust

- An inline server in an agent file from `~/.claude/agents/`, in one you pass with `--agents` or the SDK `agents` option, or in one that managed settings supplies

As of v2.1.153, the MCP restrictions that apply to the main session also cover servers declared in subagent frontmatter:

- `--strict-mcp-config` and `--bare`
- **Enterprise managed MCP configuration**
- `allowedMcpServers` and `deniedMcpServers` **policies**

When one of these blocks a server, Claude Code skips it and shows a warning naming the blocked servers.

Managed-settings restrictions apply to every subagent regardless of how it is defined. `--strict-mcp-config` doesn't filter servers you pass inline via `--agents` or the SDK `agents` option, since those are explicit caller input.

Permission modes

Set `permissionMode` to choose the permission mode a subagent runs in. Use the modes' config values, so Manual mode is `default`. If you leave it unset, the subagent inherits the main conversation's mode, which starts as **auto mode** on Pro, Max, and Team plans unless your settings or your organization change it. Setting it overrides that mode, except in the cases described below.

Mode	Behavior
<code>default</code>	Manual mode: prompts for permission
<code>acceptEdits</code>	Auto-accept file edits and common filesystem commands for paths in the working directory or <code>additionalDirectories</code>
<code>auto</code>	<u>Auto mode</u> : a background classifier reviews commands and protected-directory writes

<code>bypassPermissions</code>	Skip permission prompts
<code>plan</code>	Plan mode (read-only exploration)

⚠ Use `bypassPermissions` with caution. It skips permission prompts, allowing the subagent to execute operations without approval, including writes to `.git` , `.config/git` , `.claude` , `.vscode` , `.idea` , `.husky` , `.cargo` , `.devcontainer` , `.yarn` , and `.mvn` .

Even in this mode, the [actions no mode auto-approves](#) still apply. See [permission modes](#) for details.

If the parent uses `bypassPermissions` or `acceptEdits` , this takes precedence and can't be overridden. If the parent uses [auto mode](#), the subagent inherits auto mode and any `permissionMode` in its frontmatter is ignored: the classifier evaluates the subagent's tool calls with the same block and allow rules as the parent session.

If bypass mode is disabled by [permissions.disableBypassPermissionsMode](#) , Claude Code ignores `permissionMode: bypassPermissions` in the frontmatter and the subagent runs with the parent session's mode. Before v2.1.223, Claude Code applied the frontmatter mode even with bypass disabled.

Preload skills into subagents

Use the `skills` field to inject skill content into a subagent's context at startup. This gives the subagent domain knowledge without requiring it to discover and load skills during execution.

- `api-conventions`
- `error-handling-patterns`

Implement API endpoints. Follow the conventions and patterns from the preload

The full content of each listed skill is injected into the subagent's context at startup. This field controls which skills are preloaded, not which skills the subagent can access: without it, the subagent can still discover and invoke project, user, and plugin skills through the Skill tool during execution. To prevent a subagent from invoking skills entirely, omit `Skill` from the `tools` list or add it to `disallowedTools`.

You can't preload skills that set `disable-model-invocation: true`, since preloading draws from the same set of skills Claude can invoke. This includes the bundled `/verify` skill: only you can run it, so it can't be preloaded either.

If a listed skill is missing or disabled, for example by your organization's policy, Claude Code skips it and logs a warning to the debug log.

❗ This is the inverse of [running a skill in a subagent](#). With `skills` in a subagent, the subagent controls the system prompt and loads skill content. With `context: fork` in a skill, the skill content is injected into the agent you specify. Both use the same underlying system.

Enable persistent memory

The `memory` field gives the subagent a persistent directory that survives across conversations. The subagent uses this directory to build up knowledge over time, such as codebase patterns, debugging insights, and architectural decisions.

You are a code reviewer. As you review code, update your agent memory with patterns, conventions, and recurring issues you discover.

Choose a scope based on how broadly the memory should apply:

Scope	Location	Use when
user	<code>~/.claude/agent-memory/<name-of-agent>/</code>	the subagent should remember learnings across all projects
project	<code>.claude/agent-memory/<name-of-agent>/</code>	the subagent's knowledge is project-specific and shareable via version control
local	<code>.claude/agent-memory-local/<name-of-agent>/</code>	the subagent's knowledge is project-specific but shouldn't be checked into version control

Subagent memory is part of **auto memory**: if you turn auto memory off, with the `autoMemoryEnabled` setting or `CLAUDE_CODE_DISABLE_AUTO_MEMORY`, the `memory` field has no effect and the subagent launches without the memory instructions or the memory tool access described below.

When memory is enabled:

- The subagent's system prompt includes instructions for reading and writing to the memory directory.
- The subagent's system prompt also includes the first 200 lines or 25KB of `MEMORY.md` in the memory directory, whichever comes first, with instructions to curate `MEMORY.md` if it exceeds that limit.
- Read, Write, and Edit tools are automatically enabled so the subagent can manage its memory files.

check your memory for patterns you've seen before."

- Ask the subagent to update its memory after completing a task: "Now that you're done, save what you learned to your memory." Over time, this builds a knowledge base that makes the subagent more effective.
- Include memory instructions directly in the subagent's markdown file so it proactively maintains its own knowledge base:

Update your agent memory as you discover codepaths, patterns, library locations, and key architectural decisions. This builds up institutional knowledge across conversations. Write concise notes about what you found and where.

Conditional rules with hooks

For more dynamic control over tool usage, use `PreToolUse` hooks to validate operations before they execute. This is useful when you need to allow some operations of a tool while blocking others.

This example creates a subagent that only allows read-only database queries. The `PreToolUse` hook runs the script specified in `command` before each Bash command executes:

```
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-query.sh"
---
```

Claude Code passes hook input as JSON via stdin to hook commands. The validation script reads this JSON, extracts the Bash command, and exits with code 2 to block write operations:

```
#!/bin/bash
# ./scripts/validate-readonly-query.sh

INPUT=$(cat)
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

# Block SQL write operations (case-insensitive)
if echo "$COMMAND" | grep -iE '\b(INSERT|UPDATE|DELETE|DROP|CREATE|ALTER|TRU
  echo "Blocked: Only SELECT queries are allowed" >&2
  exit 2
fi

exit 0
```

On macOS and Linux, make the script executable, or the hook fails instead of blocking anything:

```
chmod +x ./scripts/validate-readonly-query.sh
```

See [hook input](#) for the complete input schema and [exit codes](#) for how exit codes affect behavior. On Windows, write hook scripts in PowerShell and add `shell: powershell` to the hook entry as shown in [running hooks in PowerShell](#).

Disable specific subagents

You can prevent Claude from using specific subagents by adding them to the `deny` array in your [settings](#). Use the format `Agent(subagent-name)` where `subagent-name` matches the subagent's name field.

```
{
  "permissions": {
    "deny": ["Agent(Explore)", "Agent(my-custom-agent)"]
  }
}
```

This works for both built-in and custom subagents. You can also use the `--disallowedTools` CLI flag:

```
claude --disallowedTools "Agent(Explore)"
```

See [Permissions documentation](#) for more details on permission rules.

Define hooks for subagents

Subagents can define [hooks](#) that run during the subagent's lifecycle. There are two ways to configure hooks:

- **In the subagent's frontmatter:** define hooks that run only while that subagent is active

Hooks from settings files, managed policy settings, and plugins all apply inside subagents, so a `PreToolUse` hook in `settings.json` also runs before every tool a subagent uses.

Hooks in subagent frontmatter

Define hooks directly in the subagent's markdown file. These hooks only run while that specific subagent is active and are cleaned up when it finishes.

❗ Frontmatter hooks fire when the agent is spawned as a subagent through the Agent tool or an @-mention, and when the agent runs as the main session via `--agent` or the `agent` setting. In the main-session case they run alongside any hooks defined in `settings.json`.

To let a project-level subagent's frontmatter hooks run, accept the workspace trust dialog for the folder that contains the agent file. Hooks from user-level subagents in `~/.claude/agents/` and from definitions you pass with `--agents` run without this step. If you added a folder with `--add-dir` from outside your trusted workspace's repository, trust that folder separately: its `.claude/agents/` hooks don't inherit the workspace's grant.

Until you trust the folder, the subagent still runs, but Claude Code skips its frontmatter hooks and logs an error to the debug log explaining how to trust the folder. This is a stricter rule than the one for hooks in settings files: trusting a parent folder isn't enough, and a `-p` session doesn't count as trusted. What runs before you trust a folder compares the two. Before v2.1.218, frontmatter hooks could run from folders you hadn't trusted, including in non-interactive sessions.

All hook events are supported. The most common events for subagents are:

Event	Matcher input	When it fires
<code>PreToolUse</code>	Tool name	Before the subagent uses a tool

This example validates Bash commands with the `PreToolUse` hook and runs a linter after file edits with `PostToolUse` :

```
---
name: code-reviewer
description: Review code changes with automatic linting
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-command.sh $TOOL_INPUT"
  PostToolUse:
    - matcher: "Edit|Write"
      hooks:
        - type: command
          command: "./scripts/run-linter.sh"
---
```

When the agent is invoked as a subagent, `Stop` hooks in frontmatter are automatically converted to `SubagentStop` events.

Project-level hooks for subagent events

Configure hooks in `settings.json` that respond to subagent lifecycle events in the main session.

Event	Matcher input	When it fires
<code>SubagentStart</code>	Agent type name	When a subagent begins execution
<code>SubagentStop</code>	Agent type name	When a subagent completes

with `^` and `$`, as in `^my-plugin:db-agent$`, to match only that agent.

This example runs a setup script only when the `db-agent` subagent starts, and a cleanup script when any subagent stops:

```
{
  "hooks": {
    "SubagentStart": [
      {
        "matcher": "db-agent",
        "hooks": [
          { "type": "command", "command": "./scripts/setup-db-connection.sh" }
        ]
      }
    ],
    "SubagentStop": [
      {
        "hooks": [
          { "type": "command", "command": "./scripts/cleanup-db-connection.sh" }
        ]
      }
    ]
  }
}
```

A hyphenated matcher like `db-agent` matches exactly on Claude Code v2.1.195 or later. On earlier versions it is evaluated as an unanchored regular expression and also fires for any agent type that contains it, such as `prod-db-agent`; anchor it as `^db-agent$` on those versions.

See [Hooks](#) for the complete hook configuration format.

Claude automatically delegates tasks based on the task description in your request, the `description` field in subagent configurations, and current context. To encourage proactive delegation, include phrases like “use proactively” in your subagent’s description field.

Invoke subagents explicitly

When automatic delegation isn’t enough, you can request a subagent yourself. Three patterns escalate from a one-off suggestion to a session-wide default:

- **Natural language:** name the subagent in your prompt; Claude decides whether to delegate
- **@-mention:** guarantees the subagent runs for one task
- **Session-wide:** the whole session uses that subagent’s system prompt, tool restrictions, and model via the `--agent` flag or the `agent` setting

For natural language, there’s no special syntax. Name the subagent and Claude typically delegates:

```
Use the test-runner subagent to fix failing tests
Have the code-reviewer subagent look at my recent changes
```

@-mention the subagent. Type `@` and pick the subagent from the typeahead, the same way you @-mention files. This ensures that specific subagent runs rather than leaving the choice to Claude:

```
@"code-reviewer (agent)" look at the auth changes
```

Your full message still goes to Claude, which writes the subagent’s task prompt based on what you asked. The @-mention controls which subagent Claude invokes, not what

plugin organizes agents into subagents. Named background subagents currently running in the session also appear in the typeahead, showing their status next to the name.

You can also type the mention manually without using the picker: `@agent-<name>` for local subagents, or `@agent-` followed by the scoped name for plugin subagents, for example `@agent-my-plugin:code-reviewer`. While you type this form the typeahead shows file matches rather than agents. The agent mention still resolves when you submit.

Run the whole session as a subagent. Pass `--agent <name>` to start a session where the main thread itself takes on that subagent's system prompt, tool restrictions, and model:

```
claude --agent code-reviewer
```

The subagent's system prompt replaces the default Claude Code system prompt entirely, the same way `--system-prompt` does. `CLAUDE.md` files and project memory still load through the normal message flow. The agent name appears as `@<name>` in the startup header so you can confirm it's active.

This works with built-in and custom subagents, and the choice persists when you resume the session: Claude Code restores the agent's system prompt, tool restrictions, and model along with the conversation. If the agent no longer exists when you resume, the session continues with the default tools and system prompt and shows a **warning naming the agent**.

For a plugin-provided subagent, you can pass only the agent name and Claude Code finds it:

```
claude --agent security-reviewer
```

If the plugin places the agent in a subfolder of its `agents/` directory, include the subfolder in the scoped name, for example `claude --agent my-plugin:review:security`.

To make it the default for every session in a project, set `agent` in `.claude/settings.json`:

```
{
  "agent": "code-reviewer"
}
```

The CLI flag overrides the setting if both are present.

Run subagents in foreground or background

Subagents can run in the foreground or the background:

- **Foreground subagents** block the main conversation until complete. Permission prompts are passed through to you as they come up.
- **Background subagents** run concurrently while you continue working. When a background subagent reaches a tool call that needs permission, Claude Code surfaces the prompt in your main session and names the subagent that is asking. Approve to let the subagent continue, or press Esc to deny that one tool call without stopping the subagent. Before v2.1.186, background subagents auto-denied any tool call that would have prompted.

For each subagent Claude spawns with the Agent tool, Claude Code picks foreground or background from the first of these cases that applies:

when a teammate sets `run_in_background: true` .

- If you set `CLAUDE_CODE_DISABLE_BACKGROUND_TASKS` to `1` , Claude Code runs the subagent in the foreground, in every kind of session and whether or not fork mode is on.
- Where **fork mode** is on, as it is by default in an interactive session, Claude Code runs the subagent in the background, forks and non-fork subagents alike, and Claude can't ask for the foreground.
- Where fork mode is off, Claude runs the subagent in the background by default and in the foreground when it needs the result before continuing. Fork mode is off in **non-interactive mode** with `-p` and in the Agent SDK unless you turn it on. To keep a particular subagent in the background even when Claude wants the result, set its frontmatter `background` field to `true` .

For a skill with `context: fork` , Claude Code follows the rules in **Run skills in a subagent** instead, whether or not fork mode is on.

Background subagents run with a **smaller built-in tool set** than foreground subagents, except for conversation forks, and they surface every permission prompt in your main session. When you answer one of those prompts with a choice that lasts beyond that one tool call, such as a grant that lasts for the rest of the session, Claude Code applies your answer to the whole session, including your main conversation.

A background subagent's results reach Claude as a completion notification in a later turn. Claude waits for that notification before reporting the subagent's results, and if you ask about progress first, it reports that the subagent is still running. Before v2.1.211, Claude sometimes reported results for a background subagent that hadn't finished.

You can also steer this yourself:

- Where fork mode is off, ask Claude to run a task in the background or in the foreground
- Press **Ctrl+B** to background a running task

and, except in screen reader mode, shows `/tasks` to see subagents in the footer for 30 seconds. During those 30 seconds, run `/tasks` and press `Enter` on the subagent to open its transcript. Before v2.1.232, Claude Code kept the row for 30 seconds after the subagent finished, the same as a failed one, and showed no footer hint.

- When a subagent fails or you stop it, Claude Code keeps its row for 30 seconds. To clear the row sooner, select it and press `x`.

A background subagent that completes stays listed in `/tasks`, marked done and sorted below running work, for the same window as the footer hint above. Its detail view stays open when the subagent finishes. Subagents that fail or that you stop leave the list. Before v2.1.208, a completed subagent left the list the moment it finished and its detail view closed.

Subagent names

Claude can give a subagent a name by passing a `name` parameter on the Agent tool call, and may do so on its own, without asking you first. The name makes the subagent addressable: Claude can message or resume it by name after it finishes.

In an interactive session with agent teams enabled, a subagent that Claude spawns from the main conversation with a `name` launches as a teammate instead, unless the call is a fork or passes `isolation` on the call itself. An `isolation` value in the subagent's frontmatter doesn't prevent it, and the teammate then runs in the main session's working directory. See How Claude starts agent teams.

API errors in subagents

As of v2.1.199, a subagent whose run ends on an API error, such as a usage limit or a repeated server error, reports that failure back to Claude instead of returning the error text as if it were the subagent's findings. What Claude receives depends on where the subagent ran:

API error, followed by the error detail. In v2.1.199, a rate limit, overload, or server error that cut off the tool-calls-only shape returned an empty partial result containing only the cut-off note instead.

- **Background:** the subagent is marked failed, and the message Claude receives when it ends names the API error and includes the subagent's last output, so partial work isn't lost.

When you configure a **fallback model chain** and a subagent encounters a failure the chain covers, such as its model being unavailable, Claude Code switches the subagent to the first model in the chain that accepts the request. The subagent keeps working instead of ending on the error.

Once the underlying API error clears, ask Claude to retry the task or **resume the subagent**.

Subagent output scanning

Claude Code scans each subagent's final report before Claude reads it. A subagent may have read files, web pages, or command output you never reviewed, and text from those sources can carry instructions aimed at the main conversation. The scan never removes or rewords anything; it makes two kinds of change you may notice in a report:

- **Backslash insertion:** the scan inserts a backslash into text that imitates Claude Code's own output, such as a `<system-reminder>` tag or a line starting with `Human:` or `Assistant:`, so the imitation reads as ordinary text instead of being mistaken for part of the conversation.
- **Marker line:** the scan prepends a line starting with `[harness: subagent output matched instruction-shaped pattern(s):` when the report imitates a tag like `<system-reminder>` or mentions permission settings such as `bypassPermissions` or `--dangerously-skip-permissions`. Permission-setting mentions get the marker line, but the text itself stays as written.

ⓘ Subagent output scanning requires Claude Code v2.1.210 or later.

Common patterns

Isolate high-volume operations

One of the most effective uses for subagents is isolating operations that produce large amounts of output. Running tests, fetching documentation, or processing log files can consume significant context. By delegating these to a subagent, the verbose output stays in the subagent's context while only the relevant summary returns to your main conversation.

```
Use a subagent to run the test suite and report only the failing tests
with their error messages
```

Run parallel research

For independent investigations, spawn multiple subagents to work simultaneously:

```
Research the authentication, database, and API modules in parallel using
separate subagents
```

Each subagent explores its area independently, then Claude synthesizes the findings. This works best when the research paths don't depend on each other.

⚠ When subagents complete, their results return to your main conversation. Running many subagents that each return detailed results can consume significant context.

For multi-step workflows, ask Claude to use subagents in sequence. Each subagent completes its task and returns results to Claude, which then passes relevant context to the next subagent.

Use the code-reviewer subagent to find performance issues, then use the optimizer subagent to fix them

Choose between subagents and main conversation

Use the **main conversation** when:

- The task needs frequent back-and-forth or iterative refinement
- Multiple phases share significant context, such as planning, implementation, and testing
- You're making a quick, targeted change
- Latency matters. A subagent that isn't a **fork** starts fresh and may need time to gather context

Use **subagents** when:

- The task produces verbose output you don't need in your main context
- You want to enforce specific tool restrictions or permissions
- The work is self-contained and can return a summary

Consider **Skills** instead when you want reusable prompts or workflows that run in the main conversation context rather than isolated subagent context.

For a question about something already in your conversation, use **/btw** instead of a subagent. It sees your full context but has no tool access, and the answer isn't added to history.

returns one summary. A fork at the limit keeps `Agent` in its inherited tool list, but the tool returns an error instead of spawning.

Nested subagents suit a delegated task that itself splits into parallel subtasks, such as a reviewer subagent that dispatches a verifier per finding, so the intermediate output never reaches your main conversation. Only the top-level subagent's summary returns to you.

To change the limit, set `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH` to the number of subagent layers you want below your main conversation. For example, this entry in `settings.json` caps nesting at two layers:

```
{
  "env": {
    "CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH": "2"
  }
}
```

With this value, your subagents can delegate to a second layer of their own, and that second layer can't delegate further. Set `1` to turn nesting off.

A nested subagent is configured the same way as a top-level one and resolves from the same scopes. To keep one subagent from spawning while nesting is on, such as a reviewer that should stay read-only, omit `Agent` from its tools list or add it to `disallowedTools`.

Claude Code shows nested subagents as a tree in the subagent panel below the prompt input and marks each row that still has descendants in the panel with a `(+N)` count of them. Open a row to see that subagent's siblings and direct children with a path back to `main`.

own unless you raised it; v2.1.219 raised the default to three.

Concurrent subagent limit

Two limits control subagent use, each with its own variable: this one stops Claude from spawning more subagents while too many are running, and the **depth limit** caps how deeply subagents nest. There's no limit on the total number of subagents Claude can spawn over a session.

By default, when 20 subagents are running in a session, spawning another with the Agent tool fails with `Concurrent subagent limit reached`, and the error tells Claude not to retry. Spawning succeeds again when the running count drops below the limit. To change the limit, set `CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS` to any positive whole number. Sessions with **ultracode** active are exempt: the limit isn't enforced there. Requires Claude Code v2.1.217 or later.

The limit blocks only subagents Claude spawns with the Agent tool, but other runs occupy the same slots:

- An in-session fork you start with `/subtask` takes a slot while it runs and is never blocked by the limit.
- **Resuming a subagent** that already finished takes a fresh slot without checking the limit, so resumes can push the running count past it.

Agents that other features run, such as **workflow** agents and **agent team** teammates, follow their own limits instead.

conversation history, the skills you've already invoked, or the files Claude has already read. Claude composes a delegation message that summarizes the task, and the subagent works from there. The exception is a **fork**, which inherits the parent conversation instead of starting fresh.

A non-fork subagent's initial context contains:

- **System prompt:** the agent's own prompt plus environment details that Claude Code appends, not the full Claude Code system prompt. Custom subagents define theirs in the **markdown body** or `prompt` field. Built-in agents have predefined prompts.
- **Task message:** the delegation prompt Claude writes when it hands off the work.
- **CLAUDE.md files:** every level of the **CLAUDE.md hierarchy** the main conversation loads, including `~/.claude/CLAUDE.md`, project rules, `CLAUDE.local.md`, and managed policy files. The built-in Explore and Plan agents skip this.
- **Git status:** a snapshot taken at the start of the parent session. Absent when the working directory isn't a Git repository or when `includeGitInstructions` is `false`. Explore and Plan skip it regardless.
- **Preloaded skills:** full content of any skill named in the agent's **skills field**. Built-in agents don't preload skills.
- **Sibling roster:** a system reminder listing `main` and every other named agent in the session, each a valid `to` value for `SendMessage`. Requires Claude Code v2.1.206 or later. The roster appears only when the subagent's tools include `SendMessage` and at least one other agent has a name, whether Claude named it when spawning it or it runs as an **agent team** teammate. It is a snapshot taken when the subagent starts, so agents named later don't appear.

Explore and Plan are the only subagents that omit CLAUDE.md and git status. There is no frontmatter field or per-agent setting to change which agents skip them.

The main conversation reads Explore and Plan results with full CLAUDE.md context, so most rules don't need to reach the subagent itself. If a rule must, such as "ignore the

shape its responses, except in a **fork**.

- **Auto memory:** the main conversation's **auto memory** isn't loaded. To give a subagent persistent memory of its own, use the **memory field**.
- **Context window size:** a subagent's context window is sized by its own model, not the parent's. Delegating to a model with a smaller window gives that subagent the smaller window.

Resume subagents

Each subagent invocation creates a new instance rather than continuing an earlier one. To continue an existing subagent's work instead of starting over, ask Claude to resume it.

Resumed subagents retain their full conversation history, including all previous tool calls, results, and reasoning. The subagent picks up exactly where it stopped rather than starting fresh.

- When a subagent completes, Claude receives its agent ID. The built-in Explore and Plan agents are one-shot and return no agent ID, so they can't be resumed; use `general-purpose` or a custom subagent when you need to continue the work.
- When a subagent stops at its `maxTurns` limit, Claude Code marks the returned output as partial. For subagents that return an agent ID, Claude Code also notes in the result that Claude can message the subagent to continue from where it stopped.

Claude uses the `SendMessage` tool with the agent's ID or name as the `to` field to resume it. `SendMessage` doesn't require **agent teams** to be enabled; only structured team-protocol messages such as `shutdown_request` and `plan_approval_response` do. Beyond subagents and teammates, in sessions where cross-session messaging is enabled, Claude can use the same tool to message **your other Claude Code sessions**, on this machine or **beyond it**.

To resume a subagent, ask Claude to continue the previous work:

[Claude resumes the subagent with full context from previous conversation]

A completed subagent that receives a `SendMessage` auto-resumes in the background without a new `Agent` invocation. The same applies to a subagent that Claude stopped with the `TaskStop` tool.

As of v2.1.191, a subagent you stopped yourself, with `x` in `/tasks` or an SDK `stop_task` request, doesn't auto-resume. The `SendMessage` call returns a refusal telling Claude the agent was cancelled. While **that subagent's row is still in the subagent panel**, type into its transcript to resume it yourself, which clears the stop so later `SendMessage` calls can auto-resume it again.

Resuming starts a new run of the agent under the same ID, so a subagent that had already failed or completed shows as running again in the task list and in the Agent SDK's task events. Before v2.1.205, it kept showing its earlier failed or completed status while the resumed run was working.

As of v2.1.199, `SendMessage` checks that a name still refers to the same agent it reached earlier in the conversation. If a newer agent has taken the name, such as a re-spawned background agent that reused it, Claude Code refuses the send rather than delivering it to the wrong agent, and the error reports which agent the name now reaches so Claude can retarget. To reach the earlier agent while it's still running, Claude addresses it by the agent ID it received when it spawned that agent. The check is scoped to the current conversation and resets on `/clear`.

As of v2.1.198, a subagent treats messages from the agent that launched it as normal task direction, including mid-task course corrections, and acts on them within its own permission settings. Two limits still hold regardless of who sent the message: no message from any agent counts as your approval for a pending permission prompt, and no agent message can change a subagent's permission settings, `CLAUDE.md`, or configuration. Only the permission system or your own messages can grant approval.

Subagent transcripts persist independently of the main conversation.

- **Main conversation compaction:** when the main conversation compacts, subagent transcripts are unaffected. They're stored in separate files.
- **Session persistence:** subagent transcripts persist within their session. You can [resume a subagent](#) after restarting Claude Code by resuming the same session.
- **Automatic cleanup:** Claude Code deletes subagent transcripts after the `cleanupPeriodDays` retention period, 30 days by default, following the [retention sweep rules](#).

Auto-compaction

Subagents support automatic compaction using the same logic as the main conversation. Compaction triggers under the same conditions, and

`CLAUDE_AUTOCOMPACT_PCT_OVERRIDE` applies to subagents as well. See [environment variables](#) for when the override takes effect.

Compaction events are logged in subagent transcript files:

```
{
  "type": "system",
  "subtype": "compact_boundary",
  "compactMetadata": {
    "trigger": "auto",
    "preTokens": 167189
  }
}
```

The `preTokens` value shows how many tokens were used before compaction occurred.

subagent instead, otherwise `/fork` copies the whole session into a new background session.

A fork is a subagent that inherits the entire conversation so far instead of starting fresh. This drops the input isolation that subagents otherwise provide: a fork sees the same system prompt, tools, model, and message history as the main session, so you can hand it a side task without re-explaining the situation. The fork's own tool calls still stay out of your conversation and only its final result comes back, so your main context window stays clean. Use a fork when any other subagent would need too much background to be useful, or when you want to try several approaches in parallel from the same starting point.

Claude starts a fork by requesting the `fork` subagent type through the Agent tool. You control whether it can with **fork mode**, which is on by default in interactive sessions.

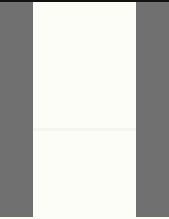
You can start a fork yourself with `/subtask` followed by a task, whether or not fork mode is on. On v2.1.161 through v2.1.211 the command is `/fork`. Claude Code names the fork from the first words of the task. The following example forks the conversation to draft test cases while you continue with the implementation in the main session:

```
/subtask draft unit tests for the parser changes so far
```

The fork appears in a panel below your prompt and runs in the background while you keep working. When it finishes, its result arrives as a message in your main conversation. The next section covers the panel controls for watching and steering forks while they run.

Observe and steer running forks

Running forks appear in a panel below the prompt input, with one row for the main session and one for each fork.



Use these keys to interact with the panel:

Key	Action
<code>↑ / ↓</code>	Move between rows
<code>Enter</code>	Open the selected fork's transcript and send it follow-up messages
<code>x</code>	Stop the selected fork if it's running, or dismiss its row if it's no longer running. On the main session row, or on the row of the fork whose transcript you opened with <code>Enter</code> , <code>x</code> types into the prompt instead
<code>Esc</code>	Return focus to the prompt input

With a fork's or subagent's transcript open, follow-up messages and skills go to that agent, but built-in commands still run in your main conversation. As of v2.1.199, typing `/model` or `/fast` in that view shows a notice that it changes the main conversation's model or fast mode, not the viewed agent's, instead of running it silently.

How forks differ from other subagents

A fork inherits everything the main session has at the moment it spawns. Any other subagent starts fresh from its definition.

	Fork	Non-fork subagent
Context	Full conversation history	Fresh context with the prompt you pass
System prompt and tools	Same as main session	From the subagent's <u>definition file</u> , <u>filtered for background runs</u>
Model	Same as main session	From the subagent's <code>model</code> field
Permissions	Prompts surface in your terminal	<u>Prompts surface in your main session</u> when running in the background
Prompt cache	Shared with main session	Separate cache



When Claude spawns a fork through the Agent tool, it can pass `isolation`:

`"worktree"` so the fork's file edits are written to a separate git worktree instead of your checkout. A fork can't spawn further forks.

Turn fork mode on or off

Claude Code turns fork mode on by default in interactive sessions and leaves it off by default in **non-interactive mode** with `-p` and in the Agent SDK. The interactive default requires Claude Code v2.1.232 or later. On earlier versions, set

`CLAUDE_CODE_FORK_SUBAGENT` to `1` to turn fork mode on.

You can tell fork mode is on from how Claude Code handles the Agent tool:

- Claude can spawn a fork by requesting the `fork` subagent type. When Claude doesn't request a type, it gets the **general-purpose** subagent, if the session still has that type. Subagents spawned from a definition, such as Explore, work as usual.
- Claude Code runs the subagents Claude spawns in the background, forks and non-fork subagents alike, apart from the **cases that stay in the foreground**. Claude Code also removes the Agent tool's `run_in_background` parameter, so Claude can't ask for the foreground.

Set the `CLAUDE_CODE_FORK_SUBAGENT` environment variable to override the defaults:

- `1` turns fork mode on in non-interactive mode and the Agent SDK as well
- `0` turns fork mode off in every kind of session

To keep fork mode on but stop Claude from spawning forks, **deny the `fork` subagent type** with an `Agent(fork)` rule. Claude Code still runs the subagents Claude spawns in the background, apart from the same **cases that stay in the foreground**.

💡 **Best practices:**

- **Design focused subagents:** each subagent should excel at one specific task
- **Write specific descriptions:** Claude uses the description to decide when to delegate
- **Limit tool access:** grant only necessary permissions for security and focus
- **Check into version control:** share project subagents with your team

Code reviewer

A read-only subagent that reviews code without modifying it. This example shows how to design a focused subagent with limited tool access that excludes Edit and Write, and a detailed prompt that specifies exactly what to look for and how to format output.

```
model: innerit
```

```
---
```

You are a senior code reviewer ensuring high standards of code quality and security.

When invoked:

1. Run git diff to see recent changes
2. Focus on modified files
3. Begin review immediately

Review checklist:

- Code is clear and readable
- Functions and variables are well-named
- No duplicated code
- Proper error handling
- No exposed secrets or API keys
- Input validation implemented
- Good test coverage
- Performance considerations addressed

Provide feedback organized by priority:

- Critical issues (must fix)
- Warnings (should fix)
- Suggestions (consider improving)

Include specific examples of how to fix issues.

Debugger

A subagent that can both analyze and fix issues. Unlike the code reviewer, this one includes Edit because fixing bugs requires modifying code. The prompt provides a clear workflow from diagnosis to verification.

You are an expert debugger specializing in root cause analysis.

When invoked:

1. Capture error message and stack trace
2. Identify reproduction steps
3. Isolate the failure location
4. Implement minimal fix
5. Verify solution works

Debugging process:

- Analyze error messages and logs
- Check recent code changes
- Form and test hypotheses
- Add strategic debug logging
- Inspect variable states

For each issue, provide:

- Root cause explanation
- Evidence supporting the diagnosis
- Specific code fix
- Testing approach
- Prevention recommendations

Focus on fixing the underlying issue, not the symptoms.

Data scientist

A domain-specific subagent for data analysis work. This example shows how to create subagents for specialized workflows outside of typical coding tasks. It explicitly sets `model: sonnet` for more capable analysis.

```
model: sonnet
```

```
---
```

You are a data scientist specializing in SQL and BigQuery analysis.

When invoked:

1. Understand the data analysis requirement
2. Write efficient SQL queries
3. Use BigQuery command line tools (bq) when appropriate
4. Analyze and summarize results
5. Present findings clearly

Key practices:

- Write optimized SQL queries with proper filters
- Use appropriate aggregations and joins
- Include comments explaining complex logic
- Format results for readability
- Provide data-driven recommendations

For each analysis:

- Explain the query approach
- Document any assumptions
- Highlight key findings
- Suggest next steps based on data

Always ensure queries are efficient and cost-effective.

Database query validator

A subagent that allows Bash access but validates commands to permit only read-only SQL queries. This example shows how to use `PreToolUse` hooks for conditional validation when you need finer control than the `tools` field provides.

```
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-query.sh"
---
```

You are a database analyst with read-only access. Execute SELECT queries to :

When asked to analyze data:

1. Identify which tables contain the relevant data
2. Write efficient SELECT queries with appropriate filters
3. Present results clearly with context

You cannot modify data. If asked to INSERT, UPDATE, DELETE, or modify schema

Claude Code **passes hook input as JSON** via stdin to hook commands. The validation script reads this JSON, extracts the command being executed, and checks it against a list of SQL write operations. If a write operation is detected, the script **exits with code 2** to block execution and returns an error message to Claude via stderr.

Create the validation script anywhere in your project. The path must match the

`command` field in your hook configuration:

```
INPUT=$(cat)

# Extract the command field from tool_input using jq
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

if [ -z "$COMMAND" ]; then
    exit 0
fi

# Block write operations (case-insensitive)
if echo "$COMMAND" | grep -iE '\b(INSERT|UPDATE|DELETE|DROP|CREATE|ALTER|TRU
    echo "Blocked: Write operations not allowed. Use SELECT queries only." >&2
    exit 2
fi

exit 0
```

On macOS and Linux, make the script executable:

```
chmod +x ./scripts/validate-readonly-query.sh
```

On Windows, write the validation script in PowerShell and add `shell: powershell` to the hook entry. See [running hooks in PowerShell](#).

The hook receives JSON via stdin with the Bash command in `tool_input.command`. Exit code 2 blocks the operation and feeds the error message back to Claude. See [Hooks](#) for details on exit codes and [Hook input](#) for the complete input schema.

The system prompt tells the subagent to refuse write requests, so the hook is a backstop: if the subagent attempts a write anyway, Claude Code blocks the command and the subagent sees the `Blocked: Write operations not allowed. Use SELECT queries only.` message.

- Run Claude Code programmatically with the Agent SDK for CI/CD and automation
- Use MCP servers to give subagents access to external tools and data

Was this page helpful?

 Yes

 No

Company

[Anthropic](#)

[Careers](#)

[Economic Futures](#)

[Research](#)

[News](#)

Help and security

[Availability](#)

[Status](#)

[Support center](#)

[Courses](#)[MCP connectors](#)[Customer stories](#)[Engineering blog](#)[Events](#)[Powered by Claude](#)[Service partners](#)[Startups program](#)[Privacy choices](#)[Privacy policy](#)[Disclosure policy](#)[Usage policy](#)[Commercial terms](#)[Consumer terms](#)